

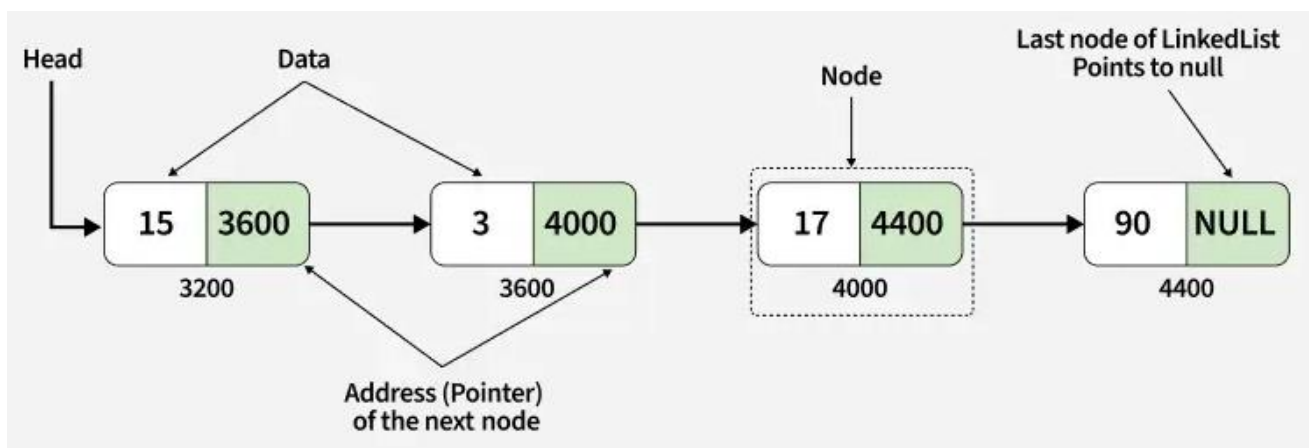
Linear - Dynamic Data Structure (Linked List)

Introduction

A **linked list** is a **flexible** data structure that consists of elements called **nodes**, each containing **data** and a **reference** to the next node. Unlike arrays, linked list data structures **do not store their elements in contiguous (متجاورة) memory locations**; instead, **each node points to the next, forming a chain**. This setup allows for efficient insertions and deletions, as you can add or remove nodes without changing the locations of other elements.

Linked List Properties

1. A linked list is a linear data structure.
2. It is define as a **collection of nodes** used for storing collections of data.
3. The **data** is stored in **non-contiguous** memory locations.
4. The access operation to the data is done sequentially.
5. Each element is a **separate object** and is called a **Node**.
6. Each node in a linked list contains two parts:
 - a. **Data field:** the actual value.
 - b. **Pointer:** **reference** (stores the address) to Next Node
7. Successive elements (العناصر المتتالية) are connected by **pointers**.
8. The last element points to **NULL** denoting the **end** of the linked-list.



Basic Terminologies of Queue

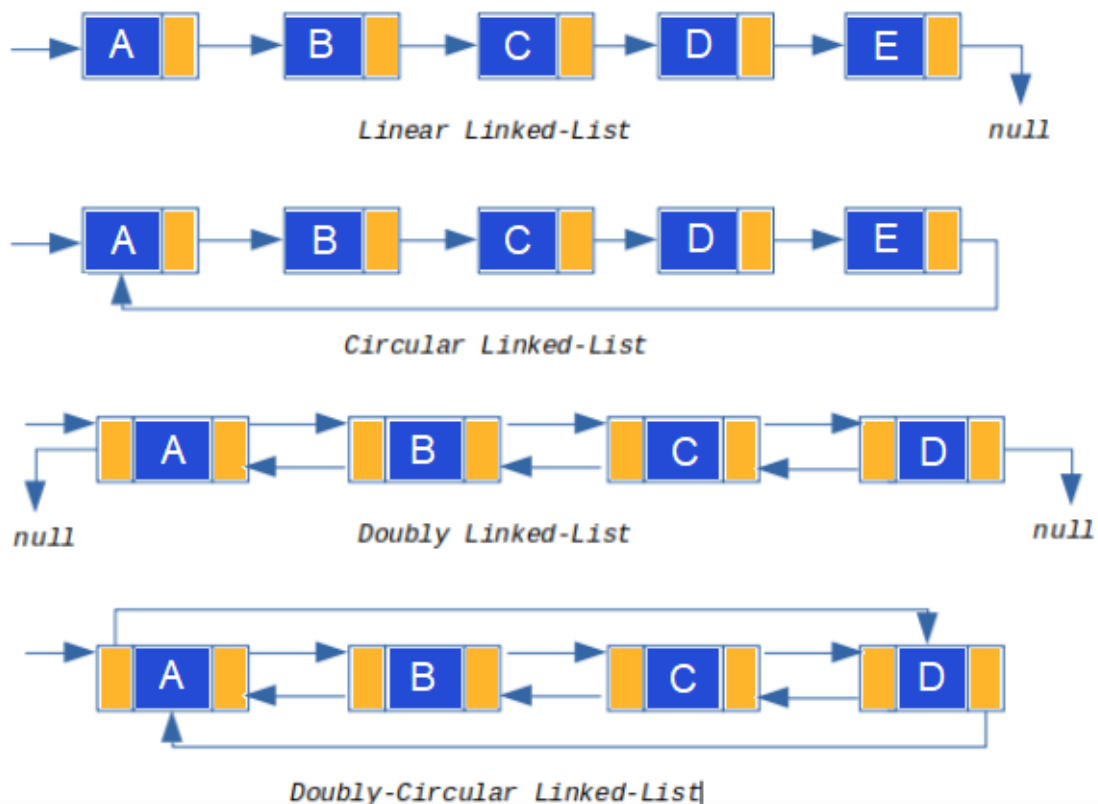
1. **Head** points to the first node of the linked-list
2. **Next** pointer of the last node is **NULL**, so if the next current node is **NULL**, we have reached the end of the linked list.

Comparing between Array & Linked-List data structure

	Array	Linked-List
Data Structure	Contiguous	Non-contiguous
Memory Allocation	Typically allocated to the whole array	Typically allocated one by one to individual elements
Insertion/Deletion	Inefficient	Efficient
Access	Random	Sequential

Types of Linked-list

There are many types of linked-lists:



Basic Operations on a Linked-List

Operation	Description
InsertAtBeginning	Attaches a new node at the start of the linked-list.
InsertAtEnd	Attaches a new node at the end of the linked-list.
InsertAtPosition	Attaches a new node at a specific position in the linked-list.
DeleteFromBeginning	Removes the Head of the linked-list and updates the Head to the next node.
DeleteFromEnd	Removes the node present at the end of the linked-list.
DeleteFromPosition	Removes a node from a given position of the linked-list.
Display	Prints all the values of the nodes present in the linked-list.
Traversal	access each element of the linked list

Creating a Node Class and Linked List Class

We have created a Node class in which we have defined a `__init__` function to initialize the node with the data passed as an argument and a reference with None because if we have only one node then there is nothing in its reference.

```
class Node:
    def __init__(self, data):
        self.data = data          # Assigns the given data to the node
        self.next = None         # Initialize the next attribute to null

class LinkedList:
    def __init__(self):
        self.head = None        # Initialize head as None
```

Implementation of 4 nodes Linked List

Example: Below is a simple example to create a singly linked list with three nodes containing integer data.

```
# Create nodes
node1 = Node(15)
node2 = Node(3)
node3 = Node(17)
node4 = Node(90)

LL = LinkedList()
# Head points to the first node
LL.head = node1
```

```
# Link nodes
node1.next = node2
node2.next = node3
node3.next = node4

# Traverse and print the linked list
current = LL.head
while current:
    print(current.data, end=" -> ")
    current = current.next
print("None")
```

Output

```
15 -> 3 -> 17 -> 90 -> None
```

Step-by-Step to find the length of the Linked-List

Finding the length of a Singly Linked-List means counting the total number of nodes:

1. Initialize a counter (**length** = 0). تهيئة العداد بأسم length
2. Start from the **head** and initialize a node pointer, **current = head**. أبدأ من بداية القائمة
 - a. Do following while **current** is not **NULL**: قم بما يلي طالما ان الـ current ليس فارغاً
 - b. Move to the next node (**current = current -> next**). حرك (او تقدم) العقدة التالية
 - c. Increment count by 1 for each node. زد العداد length لكل عقدة
3. When **current** becomes **NULL**, Return the final count. رجع العدد النهائي عندما تكون قيمة الـ current مساوية الى NULL

Algorithm of Traversal of Singly Linked-List (Iterative Approach)

The process of traversing (العبور) a singly linked-list involves **printing** the value of each node and then going on to the **next node** and print that node's value also and so on, till we reach the **last node** in the singly linked-list, whose next node points towards the **null**:

1. Initialize a temporary pointer to the head node **current = head**.
2. Check if **current = null**, then return.
3. While **current** is not **null**, print the data of the current node, then move **current** to next node.

Algorithm for searching an Element on a Linked-List

Start from the head of the linked list

1. (make **head** as the **current** node).
2. Run a loop process for check each node's data as:
 - a. If it matches the target value, return **true** (element found).

- b. Otherwise move to the next node.
 - c. Repeat until the **current** node is **NULL** because the last element points to **NULL**.
3. If no match is found, return **false**.

Inserting a new node at the beginning of a linked list

Within the `LinkedList` class, we are going to add a method to create a new node and place it at the start of the list:

```
def insertAtBeginning(self, new_data):  
    new_node = Node(new_data) # Create a new node  
    new_node.next = self.head # Next for new node becomes the current head  
    self.head = new_node      # Head now points to the new node
```

Every time you call the above method, a new node is created with your specified data. The next pointer of this new node is set to the current head of the list, which will place this node in front of the existing nodes. Finally, the newly created node is made the head of the list.

We are now going to populate this linked list with a series of words to get a better understanding of how the insertion operation works. To accomplish this, let's first create a method designed to traverse and print the list's contents:

```
def printList(self):  
    temp = self.head          # Start from the head of the list  
    while temp:  
        print(temp.data, end=' ') # Print the data in the current node  
        temp = temp.next         # Move to the next node  
    print()                   # Ensures the output is followed by a new line
```

The above method will print the contents of our linked list. Let's now proceed to use the methods we've defined to populate our list with a series of words: "the quick brown fox."

```
l1list = LinkedList()          # Create a new LinkedList instance  
# Insert each letter at the beginning using the method we created  
l1list.insertAtBeginning('fox')  
l1list.insertAtBeginning('brown')  
l1list.insertAtBeginning('quick')  
l1list.insertAtBeginning('the')
```

```
# Now 'the' is the head of the list, followed by 'quick', then 'brown' and 'fox'
l1list.printList()          # Print the list
```

The above lines of code should render the following output:

```
"the quick brown fox"
```

Inserting a new node at the end of a linked list

We will now create a method called `insertAtEnd` within the `LinkedList` class, to create a new node at the end of the list. If the list is empty, the new node will become the head of the list. Otherwise, it will be appended to the current last node in the list. Let's see how this works in practice:

```
def insertAtEnd(self, new_data):
    new_node = Node(new_data) # Create a new node
    if self.head is None:     # If the list is empty,
        self.head = new_node # make the new node the head
        return
    last = self.head
    while last.next:          # Otherwise, traverse the list to find the last node
        last = last.next
    last.next = new_node      # Make the last node point to the new node
```

The above method starts by creating a new node. It then checks if the list is empty, and if so, the new node is assigned as the head of that list. Otherwise, it traverses the list to find the last node and sets the pointer of this node to the new node.

We now need to include this method in our `LinkedList` class and use it to add a word to the end of our list. To accomplish this, amend your main function to look like this:

```
if __name__ == '__main__':
    l1list = LinkedList()
    # Insert words at the beginning
    l1list.insertAtBeginning('fox')
    l1list.insertAtBeginning('brown')
    l1list.insertAtBeginning('quick')
    l1list.insertAtBeginning('the')
```

```
# Insert a word at the end
l1ist.insertAtEnd('jumps')

# Print the list
l1ist.printList()
```

Notice that we have simply called the insertAtEnd method to print the word “jumps” at the end of the list. The above code should render the following output:

```
"the quick brown fox jumps"
```

Deleting a node from the beginning of a linked list

Deleting the first node of a linked list is easy since it simply involves pointing the head of this list to the second node. This way, the first node will no longer be a part of the list. To accomplish this, include the following method in the LinkedList class:

```
def deleteFromBeginning(self):
    if self.head is None:           # If the list is empty,
        return "The list is empty" # return this string
    self.head = self.head.next      # Otherwise, remove the head by making
                                    # the next node the new head
```

Deleting a node from the end of a linked list

To delete the last node of a linked list, we must traverse the list to find the second-last node and change its next pointer to None. This way, the last node will no longer be a part of the list. Copy and paste the following method into your LinkedList class to accomplish this:

```
def deleteFromEnd(self):
    if self.head is None:
        return "The list is empty"
    if self.head.next is None: # If there's only one node,
        self.head = None      # remove the head by making it None
        return
    temp = self.head
    while temp.next.next:     # Otherwise, go to the second-last node
        temp = temp.next
    temp.next = None          # Remove the last node by setting the next pointer of
                              # the second-last node to None
```

The above method first checks if the linked list is empty, returning a message to the user if it is. Otherwise, if the list contains a single node, that node is removed. For lists with multiple nodes, the method locates the second-last node, and its next node reference is updated to None.

Let's now update the main function to delete elements from the start and end of the linked list:

```
if __name__ == '__main__':
    llist = LinkedList()
# Insert words at the beginning
    llist.insertAtBeginning('fox')
    llist.insertAtBeginning('brown')
    llist.insertAtBeginning('quick')
    llist.insertAtBeginning('the')
# Insert a word at the end
    llist.insertAtEnd('jumps')
# Print the list before deletion
    print("List before deletion:")
    llist.printList()
# Deleting nodes from the beginning and end
    llist.deleteFromBeginning()
    llist.deleteFromEnd()
# Print the list after deletion
    print("List after deletion:")
    llist.printList()
```

The above code will print the list before and after deletion, showcasing how the insert and delete operations work in linked lists. You should see the following output after running this code:

```
List before deletion:
the quick brown fox jumps
List after deletion:
quick brown fox
```

Searching the linked list for a specific value

The final operation we will learn in this chapter is the retrieval of a specific value in the linked list. To accomplish this, the method should start at the head of the list and iterate through each

node, checking if the node's data matches the search value. Here is a practical implementation of this operation:

```
def search(self, value):
    current = self.head # Start with the head of the list
    position = 0         # Counter to keep track of the position
    while current:       # Traverse the list
        if current.data == value: # Compare the list's data
            return f"Value '{value}' found at position {position}"
        current = current.next
        position += 1
    return f"Value '{value}' not found in the list"
```

To find specific values in the linked list we've created, update your main function to include the search method we've just created:

```
if __name__ == '__main__':
    llist = LinkedList()
# Insert words at the beginning
    llist.insertAtBeginning('fox')
    llist.insertAtBeginning('brown')
    llist.insertAtBeginning('quick')
    llist.insertAtBeginning('the')
# Insert a word at the end
    llist.insertAtEnd('jumps')
# Print the list before deletion
    print("List before deletion:")
    llist.printList()
# Deleting nodes from beginning and end
    llist.deleteFromBeginning()
    llist.deleteFromEnd()
# Print the list after deletion
    print("List after deletion:")
    llist.printList()
# Search for 'quick' and 'lazy' in the list
    print(llist.search('quick')) # Expected to find
    print(llist.search('lazy')) # Expected not to find
```

The above code will render the following output:

```
List before deletion:  
the quick brown fox jumps  
List after deletion:  
quick brown fox  
Value 'quick' found at position 0  
Value 'lazy' not found in the list
```